

AI for Science in Civil Litigation: An Agent-Governed, Attorney-in-the-Loop Architecture for Legal Strategy Discovery

Alejandro Reynoso

Chief Scientist, DEFI Capital

June 10, 2026

Abstract

This paper develops an agent-governed architecture for AI-assisted civil litigation strategy within the *AI for Science* (AI4S) paradigm. Rather than treating a large language model as a legal oracle, the system places generative AI inside a closed-loop workflow in which candidate theories are proposed, ethically screened, anchored to precedent, simulated, stress-tested, reviewed by a senior-attorney layer, and then rejected, revised, or retained under explicit governance criteria. The paper is self-contained: it specifies the legal universe, case profile, agent roles, decision protocol, quantitative core, safeguards, and audit artifacts required to evaluate the architecture. A synthetic commercial dispute, *Meridian Analytics LLC v. Quantum Dynamics Corp.*, provides the running example.

The central methodological distinction is between *generation* and *authorization*. The LLM may generate theories, summarize case facts, and provide structured commentary, but it may not bypass the ethical gate, rewrite the precedent engine, ignore evidence constraints, or approve its own recommendation. Expected-value estimates are anchored in a synthetic precedent database and adjusted through deterministic rules. The continuation or stopping decision is governed by a `LitigationEvaluationAgent`; no strategy can be accepted without review by an `AttorneyAdvisorAgent` simulating a senior litigation partner.

The contribution is methodological rather than doctrinal. The system is not legal advice, does not represent actual law, and does not create an attorney-client relationship. Instead, it provides a formal blueprint for using AI in litigation and legal advisory work under controlled professional conditions: explicit case facts, synthetic jurisdictional rules, structured causes of action, ethical gates, precedent-based validation, stress tests, settlement economics, attorney-in-the-loop review, and decision logs. The resulting architecture shows how litigation-strategy exploration can be made more systematic, transparent, auditable, and accountable without displacing licensed legal judgment.

Keywords: artificial intelligence for science; civil litigation; legal strategy; agentic AI; attorney-in-the-loop; ethics gate; precedent simulation; expected value; professional governance.

1. Introduction

Litigation is, in substantial part, a disciplined discovery problem. A dispute arrives as facts, evidence, forum constraints, opposing-party conduct, and a client objective. Counsel must identify combinations of legal theories, factual arguments, evidentiary priorities, procedural moves, and settlement positions that advance the client’s lawful interests while remaining within professional and institutional bounds. That framing maps naturally onto the AI4S paradigm: a system proposes candidate solutions, tests them against structured evidence, learns from failures, and refines its search direction accordingly [1, 2, 3]. The difference from laboratory science is institutional rather than merely technical: the evidence is a case file, the validation mechanism is precedent and professional judgment, and the cost of a failed experiment may include wasted capital, damaged settlement leverage, sanctions risk, reputational harm, or an adverse judgment.

The architecture is developed through a specific synthetic commercial dispute: *Meridian Analytics LLC v. Quantum Dynamics Corp.* Meridian Analytics LLC is a software company that entered a two-year software development and licensing agreement with Quantum Dynamics Corp. Quantum paid an upfront milestone,

stopped payment at month eight, claimed the deliverables were substandard, and later launched a competing product after hiring two NDA-bound Meridian developers. Meridian asserts that it had delivery confirmations and milestone sign-off emails, that Quantum’s competing product appears to share architectural similarities with Meridian’s proprietary platform, and that Quantum falsely told prospective customers that Meridian was financially insolvent, causing the loss of two confirmed prospects. The dispute therefore presents multiple possible theories: breach of contract, trade-secret misappropriation, tortious interference, negligent or fraudulent misrepresentation, unjust enrichment, and related claims. Not all of those theories are equally strong, equally ethical, equally supported by evidence, or equally likely to survive adverse precedent.

The architecture developed here defines an AI4S grammar for litigation strategy. A synthetic legal universe defines the jurisdiction, causes of action, ethical obligations, damages caps, fee-shifting rules, discovery constraints, and a twenty-case precedent database. A case profile defines the facts, available evidence, damages categories, client goals, litigation budget, settlement floor, risk tolerance, and procedural posture. A deterministic precedent engine searches analogous cases and computes win probability, expected damages, expected value, duration, risk flags, and settlement range. An LLM generates candidate legal theories under a controlled vocabulary. An ethics gate screens those theories against simplified professional-responsibility rules grounded in familiar duties of meritorious advocacy, candor, confidentiality, fairness, and truthfulness [7]. The best admissible theory is stress-tested under adverse-precedent, evidence-gap, and hostile-venue scenarios. A settlement-versus-trial analysis compares expected litigation value with settlement economics. A simulated senior litigation partner, Diana Reyes, reviews the recommendation. Finally, a `LitigationEvaluationAgent` decides whether to continue searching, stop successfully, or halt after the maximum number of rounds.

The design is deliberately conservative. It does not ask the LLM to declare the law. It does not accept a theory merely because it is narratively persuasive. It does not assign expected value to arguments that fail the ethical screen. It does not permit an AI-generated recommendation to bypass attorney review. Instead, it separates creative legal-theory generation, ethical screening, precedent anchoring, outcome simulation, stress testing, settlement analysis, attorney judgment, and final governance into distinct modules. This separation is the central methodological contribution of the paper and reflects broader AI-risk-management principles of bounded authority, human oversight, traceability, and accountability [4, 5, 6].

The article specifies the architecture at a level sufficient for independent evaluation. It defines the legal universe, case facts, precedent engine, strategy loop, attorney-review layer, supervisory evaluation agent, settlement-versus-trial logic, stress tests, and final reporting artifacts. Implementation details are included to make the design reproducible; all assumptions, thresholds, decision rules, and limitations needed to evaluate the argument are stated in the paper itself.

The research question is therefore not whether an LLM can “solve” litigation. It cannot. The question is whether generative search can be placed inside a professional control architecture that preserves evidentiary discipline, ethical boundaries, quantitative traceability, attorney judgment, and client accountability. This paper answers that question by specifying a complete reference architecture and by making its assumptions, thresholds, and limits explicit.

The paper is organized as follows. Section 2 presents the end-to-end workflow. Section 3 describes the software architecture and the separation between the inner litigation-strategy loop and the outer evaluation agent. Section 4 specifies the computational core used for precedent matching, win-probability estimation, damages estimation, expected value, settlement range, and fragility classification. Section 5 formalizes the evaluation-agent protocol. Section 6 describes the ethical, evidentiary, numerical, and governance safeguards. Section 7 explains the LLM integration. Section 8 presents the stress-testing scenarios. Section 9 discusses contributions, limitations, and future research. The appendix records implementation artifacts and includes the workflow diagram.

The intended contribution is methodological. The paper does not claim that a synthetic precedent database can substitute for legal research, that a simple expected-value model can capture the economics of litigation,

or that an LLM can replace a licensed attorney. Instead, it demonstrates how AI can be placed inside a disciplined professional workflow: one that records facts, checks ethical constraints, anchors estimates to precedent, stress-tests recommendations, preserves attorney judgment, and maintains an auditable decision trail.

2. Workflow Overview

The workflow is a closed-loop litigation-strategy discovery procedure rather than a one-shot legal research assistant. Each cycle begins with structured case facts and a synthetic legal universe, generates five candidate theories, screens them through an ethical gate, simulates their expected outcomes using precedent and evidence, stress-tests the strongest admissible theory, evaluates settlement economics, obtains LLM feedback, generates next-cycle seeds, and sends the result to a senior-partner review layer. The outer evaluation agent then decides whether to stop or continue.

Table 1 summarizes the ten-stage workflow specified by the architecture.

Table 1: Ten-stage AI4S litigation-strategy workflow.

Stage	Description
1. Synthetic Legal Universe	Encodes jurisdictional rules, ethical obligations, claims, damages caps, fee-shifting rules, discovery constraints, and precedent records.
2. Case Profile	Defines <i>Meridian Analytics LLC v. Quantum Dynamics Corp.</i> , including facts, evidence, damages, client objectives, risk tolerance, settlement floor, and budget.
3. Baseline Simulation	Computes a breach-of-contract baseline from analogous precedent and case evidence.
4. Theory Generation	The configured LLM proposes five structured theories with causes of action, damages keys, evidence, strength, and risk.
5. Ethical Gate	The LLM screens theories under simplified professional-responsibility rules and classifies each as ETHICAL, BORDERLINE, or UNETHICAL.
6. Precedent Simulation	Deterministic functions estimate win probability, damages, expected value, duration, fee shifting, settlement range, and risk flags.
7. Stress Testing	The strongest admissible theory is tested under adverse-precedent, evidence-gap, and hostile-venue scenarios.
8. Settlement Analysis	The system compares settlement range, litigation budget, settlement floor, and net expected trial value.
9. Attorney Review	A simulated senior litigation partner reviews the recommendation for ethics, evidence sufficiency, strategy, and client alignment.
10. Evaluation Decision	<code>LitigationEvaluationAgent</code> decides whether to continue, stop successfully, or halt at the maximum-round boundary.

The workflow contains two nested loops. The inner loop performs one complete strategy-research cycle: theory generation, ethical screening, precedent simulation, stress testing, settlement analysis, feedback, and next-cycle seed generation. It is implemented by `LitigationStrategyLoop`. The outer loop, implemented by `LitigationEvaluationAgent`, governs the full process across up to three rounds, accumulates failure context, stores rejected theory names, records partner verdicts, and determines whether further search is justified.

The critical control point is the ethics-and-attorney gate. A legal theory is not accepted solely because it produces a favorable expected value. It must be supported by available evidence, survive the ethical screen, produce adequate expected value and win probability, remain non-fragile under adverse scenarios, and receive attorney approval or approval with modifications. This architecture converts AI-assisted legal strategy from

a plausible narrative exercise into a governed discovery process with explicit evidentiary, economic, and professional controls.

The workflow also imposes a hierarchy of evidence. A generated legal theory begins as a hypothesis. It gains evidentiary status only after it is expressed as a structured cause of action with specified damages keys and evidence support. It gains additional status after ethical screening. It gains further credibility if precedent-based simulation produces acceptable expected value. It becomes more robust if it survives adverse scenarios. It becomes professionally plausible only after senior-partner review. Finally, it becomes a candidate recommendation only if the evaluation agent records a `STOP_SUCCESS` decision. This hierarchy is central to the AI4S formulation: the model proposes, while governance decides.

3. Software Architecture

The architecture is organized around four layers: a synthetic legal environment, a deterministic precedent-and-outcome engine, an inner litigation strategy loop, and an outer governance agent. This separation is not merely a software preference. It is the primary mechanism by which the system preserves professional accountability. Each layer has a defined responsibility, a bounded authority, and an auditable interface to the next layer.

3.1. Synthetic Legal Universe

The `JURISDICTION`, `ETHICS_RULES`, `CAUSES_OF_ACTION`, and `PRECEDENTS` objects define the synthetic legal environment. The jurisdiction is the fictional State of Veridian. It specifies limitation periods, damages caps, discovery constraints, fee-shifting rules, jury-trial availability, and arbitration-clause enforceability. The ethics rules are a simplified representation of professional duties, including meritorious claims, candor to the tribunal, confidentiality, fairness to opposing parties, truthfulness to third parties, and the prohibition of dishonest conduct. The causes-of-action library includes breach of contract, breach of fiduciary duty, fraudulent misrepresentation, negligent misrepresentation, tortious interference, unjust enrichment, conversion, wrongful termination, hostile work environment, and trade-secret misappropriation. The precedent database contains twenty synthetic cases, each with type, facts, outcome, damages, duration, key factor, and plaintiff-strength classification.

The purpose of this legal universe is not to reproduce actual law. Its methodological role is to create a controlled environment in which LLM-generated theories can be tested against explicit rules. The LLM is not permitted to invent doctrine, modify the jurisdiction, or ignore the precedent database during simulation. This design targets a common failure mode of generative legal systems: fluent but unsupported claims about legal viability [9, 6].

3.2. Case Facts and Client Profile

The `CASE` dictionary defines the specific dispute. Meridian Analytics LLC is the plaintiff; Quantum Dynamics Corp. is the defendant. The case profile contains the core facts, evidence inventory, damages categories, client goals, risk tolerance, budget, settlement floor, arbitration status, filing deadline, jurisdiction, judge assignment, and opposing counsel profile.

The evidence inventory is especially important. It includes signed contract, milestone sign-off emails, payment records, NDA agreements, competing-product analysis, false-statement witnesses, developer departure records, absence of forensic copy evidence, absence of written defect notice, and availability of an expert witness. These facts are not merely narrative. They affect the simulation. For example, milestone

sign-off emails, NDA agreements, and false statement witnesses increase win probability, while the absence of forensic copy evidence reduces it and creates a risk flag for trade-secret claims.

The damages categories include unpaid contract balance, lost prospect revenue, trade-secret development cost, reputational harm estimate, and attorneys' fees to date. These categories define the economic stakes available to generated theories. A theory cannot claim damages in the abstract; it must specify which damages keys it relies upon.

3.3. Precedent Engine and Outcome Simulator

The deterministic computational engine is implemented by two functions. `search_precedents()` retrieves analogous synthetic precedents by cause of action, falling back to strong plaintiff precedents if no direct match exists. `simulate_theory()` converts a structured theory into a `SimulationResult` containing analogous cases, win probability, expected damages, damages range, expected value, duration estimate, fee-shifting status, settlement range, risk flags, and explanatory notes.

This layer is the computational center of the architecture. The LLM may generate a theory, but it does not directly assign the expected value used by the system. The outcome estimate is calculated through the deterministic simulator. This separation between legal-language generation and quantitative evaluation is essential for auditability. A reviewer can inspect which cause of action was used, which precedent records were selected, how evidence adjusted the win probability, how damages were estimated, and why risk flags were generated.

3.4. LitigationStrategyLoop

`LitigationStrategyLoop` executes one research cycle. It is responsible for generating theories, running the ethical gate, simulating all non-unethical theories, selecting the best admissible theory by expected value, stress-testing that theory, conducting settlement analysis, obtaining strategic feedback, and generating next-cycle seeds. The output is stored in a `LitigationCycleResult` dataclass.

The dataclass contains cycle number, generated theories, ethical verdicts, simulation results, best theory, best simulation, baseline expected value, stress results, fragility status, settlement recommendation, feedback verdict, and next seeds. This object functions as an audit record. It preserves both successful and unsuccessful candidates and prevents the process from relying on informal memory or hidden state.

3.5. AttorneyAdvisorAgent

`AttorneyAdvisorAgent` simulates Diana Reyes, a senior litigation partner with twenty-eight years of commercial litigation experience. Her role is not to generate new theories, but to review the AI-generated recommendation. She evaluates ethical compliance, evidence sufficiency, alignment with client goals, tactical realism, settlement posture, and professional defensibility. Her verdict may be `APPROVE`, `APPROVE_WITH_MODS`, or `REJECT`. She also provides rationale, required modifications, tactical recommendations, concerns, and client talking points.

This layer represents professional accountability. A theory that passes simulation may still be unsuitable if it depends on missing evidence, creates unnecessary ethical risk, exceeds client tolerance, or cannot be presented responsibly. The advisor layer is therefore a binding control rather than a stylistic flourish. In the reference architecture, the evaluation agent cannot issue `STOP_SUCCESS` unless the partner review supports the recommendation.

3.6. LitigationEvaluationAgent

LitigationEvaluationAgent is the supervisory governance layer. It controls the maximum number of rounds, stores round results, stores attorney reviews, accumulates failure context, records rejected theory names, and maintains a decision log. It sets $\text{MAX_ROUNDS} = 3$, $\text{MIN_EV} = 300000$, and $\text{MIN_WIN_PROB} = 0.50$. The heuristic stop condition requires attorney approval or approval with modifications, expected value of at least \$300,000, win probability of at least 50 percent, and non-fragility under stress testing.

The supervisory layer prevents uncontrolled search. Without it, a generative system could continue producing theories until it finds one that looks favorable or rhetorically compelling. The evaluation agent imposes round limits, expected-value thresholds, win-probability thresholds, fragility controls, attorney-review constraints, and failure-context memory. It converts AI-assisted legal exploration into a bounded professional workflow rather than an unconstrained prompt sequence.

4. Mathematical and Computational Core

The computational core defines the quantities the system is allowed to compute and compare. This formal layer is essential because LLMs are fluent in legal argument but must not be permitted to invent expected values, precedent effects, or settlement ranges after observing results. The architecture therefore specifies precedent matching, win probability, evidence adjustment, damages estimation, expected value, settlement range, net trial value, and stress fragility. The formulas are intentionally simple, but their simplicity is a governance feature: every transformation can be inspected, reproduced, and challenged.

4.1. Precedent Matching

Let $\mathcal{P}$ denote the synthetic precedent database. Each precedent $p \in \mathcal{P}$ contains a cause-of-action type, outcome, damages, duration, key factor, and strength label. For a generated theory h with primary cause of action c_h , the precedent engine selects

$$\mathcal{P}_h = \{p \in \mathcal{P} : \text{type}(p) = c_h\}. \quad (1)$$

If no direct match exists, the implementation falls back to precedents with strong plaintiff outcomes. This fallback is a pedagogical design choice rather than a claim about real legal analogy. In production, precedent retrieval would require jurisdictional hierarchy, procedural posture, factual similarity, citation validity, negative-treatment checks, and authoritative legal research.

4.2. Win-Probability Estimation

The baseline win probability for theory h is computed from the outcomes of the analogous precedent set. Let $N_h = |\mathcal{P}_h|$, let W_h be the number of plaintiff wins, and let S_h be the number of split verdicts. The initial win probability is

$$\pi_h^{(0)} = \frac{W_h + 0.5S_h}{N_h}, \quad N_h > 0. \quad (2)$$

When no precedent is available, the system applies a default prior. The initial estimate is then adjusted by the available evidence. In simplified form,

$$\pi_h = \text{clip} \left(\pi_h^{(0)} + \Delta_{\text{signoff}} + \Delta_{\text{NDA}} + \Delta_{\text{witness}} + \Delta_{\text{no defect}} - \Delta_{\text{forensic gap}}, 0.10, 0.95 \right). \quad (3)$$

In the implementation, milestone sign-off emails, NDA agreements, two or more false-statement witnesses, and lack of written defect notice increase win probability, while the absence of forensic copy evidence reduces it. The `clip` operator truncates the result to the interval $[0.10, 0.95]$. These bounds are not empirical claims. They are safeguards against spurious certainty and force the system to acknowledge residual litigation risk.

4.3. Damages Estimation

Let $\mathcal{D}_h$ denote the damages keys selected by the generated theory. The base damages amount is

$$B_h = \sum_{k \in \mathcal{D}_h} D_k, \quad (4)$$

where D_k is the amount associated with damages category k in the case profile. If no damages keys are supplied, the implementation uses a conservative fraction of total damages. The simulated damages range is then

$$L_h = 0.55B_h, \quad (5)$$

$$U_h = \min(1.40B_h, mB_h), \quad (6)$$

where m is the punitive multiplier cap in the synthetic jurisdiction. The expected damages estimate is

$$E[D_h] = \frac{L_h + U_h}{2}. \quad (7)$$

This damages model is simplified, but it serves the governance objective: economic assumptions are encoded and inspectable. The LLM does not invent the damages range; it selects damages keys, and the simulator computes the result. In a production setting, this layer would need to distinguish expectation, variance, recovery timing, collection probability, fee exposure, insurance coverage, punitive-damages standards, and tax effects.

4.4. Expected Value

The litigation expected value of theory h is

$$EV_h = \pi_h \cdot E[D_h]. \quad (8)$$

This expected value is the central ranking statistic inside the inner strategy loop. The best admissible theory in a cycle is selected by sorting simulated ethical theories by EV_h . However, expected value alone does not authorize a recommendation. The theory must also survive ethical screening, stress testing, settlement analysis, and partner review. This distinction is important in litigation finance: the numerically largest claim may be inferior to a smaller claim with stronger evidence, lower volatility, better settlement leverage, or lower reputational cost.

4.5. Settlement Range and Net Trial Value

The system derives a settlement range as a fraction of expected value:

$$\text{SettleLow}_h = 0.40EV_h, \quad \text{SettleHigh}_h = 0.70EV_h. \quad (9)$$

Let C_L denote the maximum litigation budget. The net expected trial value is

$$NEV_h = EV_h - C_L. \quad (10)$$

The settlement recommendation compares the settlement range, the client’s settlement floor, and the net trial expected value. If the high end of the settlement range meets the floor and approaches the net trial value, the system recommends settlement. If net trial value substantially exceeds the settlement range, the system recommends litigation. Otherwise, it recommends negotiation with an assertive pre-trial settlement demand. This logic does not purport to solve settlement theory. It creates a transparent bridge between expected value and client counseling.

4.6. Fragility Criterion

The stress-testing engine computes expected value under three adverse scenarios: adverse precedent, evidence gap, and hostile venue. Let $EV_h^{(A)}$, $EV_h^{(B)}$, and $EV_h^{(C)}$ denote the stressed expected values. Let F denote the client’s settlement floor. The system classifies a theory as fragile when

$$\min(EV_h^{(A)}, EV_h^{(B)}, EV_h^{(C)}) < 0.80F. \quad (11)$$

This criterion captures the governance principle that a recommendation should not depend solely on a favorable base case. If adverse precedent, missing forensic evidence, or hostile venue assumptions drive expected value too far below the client’s acceptable settlement economics, the theory should not be accepted without modification, additional discovery, or a revised settlement posture.

4.7. Governance-Constrained Selection

A cycle’s provisional best theory is selected by expected value after removing unethical theories:

$$h^* = \arg \max_{h \in \mathcal{H}_{\text{ethical}}} EV_h. \quad (12)$$

The final stopping condition is stricter:

$$\text{StopSuccess}(h^*) = \mathbf{1} [EV_{h^*} \geq 300,000 \wedge \pi_{h^*} \geq 0.50 \wedge \neg \text{Fragile}(h^*) \wedge \text{PartnerApproved}(h^*)]. \quad (13)$$

This formulation captures the core logic of the architecture. Litigation strategy is not unconstrained expected-value maximization. It is professionally governed discovery under ethical, evidentiary, economic, procedural, and human-review constraints.

5. The LitigationEvaluationAgent Decision Protocol

The `LitigationEvaluationAgent` formalizes the continuation problem. In traditional litigation strategy, the decision to continue researching, refine a theory, recommend settlement, or proceed toward litigation is often made through professional judgment supported by memoranda and discussion. The architecture does not attempt to replace that judgment. Instead, it makes the decision protocol explicit, inspectable, and auditable.

5.1. Motivation

The motivation for a separate evaluation agent is governance discipline. The component that generates theories should not be the same component that decides whether the process is complete. The theory generator is designed to explore the legal strategy space. The evaluation agent is designed to control that exploration. This separation mirrors the professional distinction between associate research, partner review, client counseling, and final litigation decision.

The agent applies four broad criteria. First, the theory must be ethically admissible. Second, it must be economically meaningful, with expected value above a minimum threshold. Third, it must be sufficiently plausible, with win probability above a minimum threshold. Fourth, it must survive stress tests and partner review. A theory that fails any of these conditions generally triggers another cycle or is rejected.

5.2. Two-Step Protocol

The decision protocol has two stages. The first is a deterministic heuristic. If the partner review is `REJECT`, the agent continues. If the partner approves or approves with modifications, expected value is at least \$300,000, win probability is at least 50 percent, and the theory is not fragile, the heuristic recommends `STOP_SUCCESS`. Otherwise, it recommends `CONTINUE`.

The second stage is a constrained LLM decision. The evaluation prompt receives the round number, maximum rounds, theory name, expected value, win probability, fragility status, settlement recommendation, LLM feedback, partner verdict, partner rationale, required modifications, and heuristic recommendation. It must return a structured decision object:

```
{
  "decision": "CONTINUE | STOP_SUCCESS | STOP_MAX_ROUNDS",
  "reasoning": "short explanation grounded in metrics and partner review",
  "failure_summary": "what to improve next cycle"
}
```

The schema is intentionally narrow. The LLM may reason about the evidence, but it may not alter the contract, expand the decision vocabulary, or conceal the failure mode. If the maximum-round boundary has been reached, the code prevents the model from continuing indefinitely by converting an attempted `CONTINUE` decision into `STOP_MAX_ROUNDS`.

5.3. Failure Context

A key feature of the protocol is failure-context accumulation. The current code records the best theory from each continuing round, the expected value, partner verdict, required modifications, AI diagnosis, and seed names for the next cycle. That failure context is injected into the next `LitigationStrategyLoop`, instructing the LLM not to regenerate failed theories and to propose stronger, more defensible alternatives. The result is a bounded search process with memory rather than a series of independent prompts.

In an AI4S framework, failure is not discarded; it becomes an experimental result. A theory that fails because forensic evidence is missing, adverse precedent is too damaging, the settlement economics are weak, or the partner would not sign the recommendation teaches the system where not to search next. The `LitigationEvaluationAgent` is the mechanism that converts failed legal strategies into structured memory.

6. Ethical, Evidentiary, and Governance Safeguards

AI-assisted litigation systems require safeguards because legal strategy is a high-stakes professional domain. An erroneous theory can waste client resources, damage settlement leverage, violate ethical duties, expose counsel to sanctions, or undermine the client’s position. The architecture therefore embeds safeguards at several levels: ethical review, structured theory representation, precedent anchoring, deterministic simulation, stress testing, attorney review, and stopping logic.

6.1. Ethical Gate

The ethical gate is mandatory. The configured LLM is instructed to act as bar ethics counsel and apply simplified versions of rules concerning meritorious claims, candor to the tribunal, fairness to opposing parties, and misconduct. Each generated theory receives a verdict: `ETHICAL`, `BORDERLINE`, or `UNETHICAL`. The simulation loop excludes `UNETHICAL` theories from expected-value analysis. Borderline theories may be simulated for diagnostic purposes, but they remain subject to attorney review and cannot pass the outer governance layer without professional justification.

This is a critical design choice. The system cannot generate an expected-value number for an unethical argument and present it as a legitimate option. In professional domains, quantification can create false legitimacy. The ethical gate prevents a theory from gaining apparent analytical status before it has cleared the minimum professional threshold [7, 6].

6.2. Structured Theory Safeguards

The theory-generation prompt requires structured JSON output. Each theory must include a name, primary cause of action, no more than two secondary causes, damages keys, two-sentence theory description, evidence relied upon, estimated strength, and estimated risk. This structure converts an open-ended legal narrative into an inspectable object.

Structured theory representation reduces ambiguity. A theory that says “pursue all available claims aggressively” is not testable. A theory that specifies trade-secret misappropriation, relies on NDA agreements and competing-product analysis, and claims trade-secret development cost is testable. The system therefore forces legal language into a computational form before simulation and preserves the assumptions that drive the result.

6.3. Precedent Anchoring

The precedent engine anchors probability estimates to analogous synthetic case outcomes. This prevents the model’s general optimism from becoming the source of the win probability. The LLM can propose a theory, but the simulator determines the initial probability using prior outcomes and then adjusts for evidence.

A production version would require a far more sophisticated legal research layer: jurisdictional hierarchy, negative-treatment checks, factual-similarity scoring, procedural posture, evidentiary admissibility, damages proof, and current-law validation. The paper specifies the architecture, not a complete legal research engine or a substitute for research under the Federal Rules of Civil Procedure and applicable law [8].

6.4. Professional-Review Safeguards

The `AttorneyAdvisorAgent` models professional skepticism. Diana Reyes reviews the proposed strategy as a senior litigation partner. She checks ethical compliance, evidence sufficiency, alignment with client goals, settlement posture, tactical next steps, remaining concerns, and how the matter should be

presented to the client. Her modifications are not discarded; they enter the failure context and shape the next cycle.

The most important governance point is that the attorney cannot be bypassed. The evaluation agent will not issue `STOP_SUCCESS` unless the partner approves or approves with modifications. A rejected theory triggers another cycle. Professional judgment is therefore a binding constraint, not a rubber stamp.

6.5. Stopping Safeguards

The maximum number of rounds prevents uncontrolled search. The expected-value threshold prevents the system from celebrating immaterial theories. The win-probability threshold prevents high-damages but low-plausibility theories from being accepted too quickly. The stress-fragility rule prevents a theory from advancing if it collapses under adverse assumptions. The attorney-review condition prevents model confidence from replacing professional accountability. Together, these controls convert AI-assisted legal research into governed strategy exploration.

7. LLM Integration

The LLM is integrated as a constrained reasoning and generation component. It is not treated as a court, a judge, a legal authority, or the final decision maker. Its primary functions are legal-theory generation, ethical screening, strategic feedback, next-cycle seed generation, senior-partner simulation, and evaluation-agent reasoning. Each function is governed by a prompt contract and expects structured output. The model is therefore an interface for bounded professional reasoning, not an independent source of legal authority.

One reference implementation uses Anthropic Claude through a centralized function. The model identifier is stored in `MODEL`:

```
MODEL = 'claude-haiku-4-5-20251001'
```

The wrapper function is:

```
ask_claude(system, user, max_tokens=1200)
```

This function sends system and user prompts to the model and returns text. The implementation initializes the model client with an environment-provided API key and uses a separate parsing utility to remove common Markdown code fences and parse JSON. This design is important because LLM outputs can be semantically rich but structurally unstable. The surrounding code enforces the interface and treats malformed output as an engineering problem rather than a legal conclusion.

The reference implementation uses six prompt families:

1. **Theory generation.** The LLM proposes exactly five legal theories, each with causes of action, damages keys, evidence relied upon, strength, and risk.
2. **Ethical review.** The LLM acts as ethics counsel and assigns `ETHICAL`, `BORDERLINE`, or `UNETHICAL` verdicts.
3. **Strategic feedback.** The LLM evaluates the selected theory after stress testing and settlement analysis, identifying the largest vulnerability and one tactical improvement.
4. **Next-cycle seed generation.** The LLM proposes two refined strategic directions based on feedback and rejected theory names.

5. **Attorney review.** The LLM simulates Diana Reyes, a senior litigation partner, and returns a professional verdict, modifications, tactical recommendations, concerns, and client talking points.
6. **Evaluation-agent decision.** The LLM helps the evaluation agent decide whether to continue, stop successfully, or halt at maximum rounds.

The most important design principle is that LLM judgment is never allowed to replace professional governance. The LLM may propose theories and produce structured commentary, but it does not compute expected value independently, does not bypass the ethics gate, does not replace precedent anchoring, and does not override attorney review. Stronger models may improve the quality of generated theories, but they do not remove the need for ethical screening, deterministic simulation, stress testing, partner approval, and audit logs. This point is central to legal deployment: model capability and professional authority are distinct categories.

8. Stress Testing Scenarios

Stress testing asks whether a candidate litigation theory remains credible when important assumptions deteriorate. A strategy that looks favorable under the base case may become weak if precedent turns adverse, forensic evidence cannot be obtained, or the assigned venue is unfavorable. The architecture therefore subjects the best theory in each cycle to three deterministic stress scenarios. The scenarios operate like sensitivity tests in litigation finance: they reveal which assumptions drive expected value and which facts must be developed before the strategy can responsibly be recommended.

The scenarios are illustrative rather than exhaustive. Their purpose is to model the governance principle that a recommendation should not be judged only in the base case. A production system would include discovery risk, motion-to-dismiss probability, summary-judgment probability, evidentiary exclusion, expert admissibility, counterclaim exposure, insurance coverage, collectability, reputational harm, and judge-specific litigation behavior. It would also distinguish legal merits from practical recovery: a meritorious claim with poor collectability, excessive cost, or unacceptable business disruption may be a weak client recommendation.

Scenario A — Adverse Precedent. The system reduces win probability by twenty percentage points. This scenario tests whether the theory remains economically meaningful if analogous case law is less favorable than initially assumed.

Scenario B — Evidence Gap. The system reduces win probability by fifteen percentage points to reflect the failure to obtain critical forensic evidence. This is especially material for trade-secret or misappropriation theories, where architectural similarity may not be enough without stronger proof of copying or improper use.

Scenario C — Hostile Venue. The system reduces win probability by twelve percentage points and lowers expected damages by twenty-five percent. This scenario approximates an unfavorable judge, skeptical jury pool, procedural headwinds, or damages discounting.

A theory is classified as fragile if the minimum stressed expected value falls below eighty percent of the client's settlement floor. Fragility does not mean that the theory has no value. It means that the theory should not proceed without modification, additional evidence, or a more cautious settlement posture. In the AI4S loop, fragility becomes structured feedback for the next cycle rather than an unrecorded concern in counsel's judgment.

9. Discussion and Limitations

The architecture should be evaluated as a governance design, not as a claim of real-world legal completeness. Its primary strength is the explicit connection between generative AI, ethical screening, precedent-based

simulation, stress analysis, settlement economics, attorney review, and decision logging. Its primary limitation is that the legal environment is synthetic and simplified. That limitation is intentional for a reference architecture: a controlled legal universe makes the governance architecture testable before it is connected to real client files, real courts, or real legal-research systems.

9.1. Contributions

The first contribution is a formal AI4S model of litigation strategy. The paper shows how observation, representation, hypothesis generation, ethical screening, precedent validation, stress testing, feedback, and revision can be implemented as a professional legal workflow. The second contribution is the separation between `LitigationStrategyLoop` and `LitigationEvaluationAgent`. This separation prevents the component that generates theories from also owning the final governance decision. The third contribution is the explicit attorney-in-the-loop layer represented by `AttorneyAdvisorAgent`. The fourth contribution is the insistence that expected value be anchored to precedent and evidence rather than accepted as an LLM assertion. The fifth contribution is the integration of settlement economics before recommendation, recognizing that the strongest legal theory is not always the best client advice. The sixth contribution is the preservation of negative results: rejected theories, partner concerns, and stress failures become structured inputs to the next round rather than disappearing from the record.

9.2. Limitations

The most important limitation is that the legal universe is synthetic. Real litigation would require actual statutes, rules of procedure, case law, evidence law, judicial tendencies, local practice, procedural posture, discovery dynamics, motion practice, privilege analysis, sanctions risk, insurance and indemnity issues, collectability, counterclaims, appeal risk, and client-specific nonfinancial objectives.

The synthetic design also means that the numerical outputs should not be read as calibrated forecasts. They are demonstrative artifacts used to test the architecture. In real deployment, probability estimates and damages estimates would require empirical validation, matter-specific factual development, attorney judgment, and documented uncertainty intervals.

A second limitation is that the attorney advisor is simulated by an LLM. This is useful for prototyping, but it is not a substitute for licensed professional review. In a real litigation practice, the system should escalate candidate theories to an attorney with responsibility for the matter before any client communication, settlement demand, pleading, or litigation step.

A third limitation is stochasticity. LLM-generated theories may vary across runs. This creates both opportunity and risk. Diversity can improve strategy search, but it also requires logging, versioning, reproducibility controls, and prompt governance. A production system would need to store prompts, responses, model versions, case-file versions, precedent-database versions, and calculation hashes. It would also need access controls, privilege protections, confidentiality safeguards, and retention policies appropriate for legal work.

A fourth limitation is that expected value is simplified. Real litigation expected value is path-dependent. It changes after pleadings, discovery, motions, depositions, expert reports, settlement conferences, dispositive rulings, trial preparation, trial, and appeal. A production system would need a multi-stage litigation tree rather than a single expected-value calculation.

9.3. Future Work

Future work should proceed along five lines. First, the synthetic precedent database should be replaced with validated legal-research pipelines that include jurisdiction, procedural posture, treatment history, and factual similarity scoring. Second, the expected-value engine should be expanded into a multi-stage decision tree that models pleading risk, discovery costs, summary judgment, settlement probability, trial probability, appeal

risk, and collectability. Third, the ethics gate should be strengthened with formal professional-responsibility checks and escalation protocols. Fourth, the attorney-review layer should be connected to actual review workflows, including privilege-preserving memoranda and partner sign-off. Fifth, the dashboard should export model cards, risk registers, strategy memos, reproducibility manifests, approval checklists, and audit bundles.

A particularly important extension is client-centered governance. Litigation strategy is not merely the maximization of expected monetary value. Clients may care about speed, confidentiality, reputation, precedent, relationships, deterrence, emotional burden, public exposure, and business continuity. A mature AI4S litigation system should incorporate those objectives explicitly and should record how they affected the recommendation.

10. Conclusion

This paper has presented an AI4S architecture for agent-governed litigation strategy. The system combines a synthetic legal universe, a structured case file, an evidence inventory, a precedent-based simulation engine, LLM-generated legal theory search, mandatory ethical screening, stress testing, settlement-versus-trial analysis, senior-partner review, and a supervisory evaluation agent. Its central design principle is that AI creativity must be embedded inside professional legal governance.

The architecture is not a legal chatbot. It is a controlled discovery loop. The LLM proposes legal theories, but does not authorize them. The ethical gate excludes theories that should not be quantified. The precedent engine anchors probability estimates to analogous synthetic outcomes. The stress tests expose fragility under adverse precedent, evidence gaps, and hostile venue conditions. The settlement analysis forces economic comparison between litigation and resolution. The attorney advisor introduces professional skepticism and cannot be bypassed. The evaluation agent coordinates these layers and records why the process continues, stops successfully, or halts at the maximum-round boundary.

The deeper point is that litigation has always been a propose-test-learn process. Lawyers generate theories, test them against facts and law, revise them after discovery, reassess after motions, and update advice as uncertainty resolves. What AI4S adds is not a replacement for that professional process, but a disciplined way to make it more systematic. The model can search more broadly, evaluate alternatives in parallel, preserve negative evidence, and make failure analysis explicit rather than implicit.

The running dispute makes this claim operational. It shows how a commercial case can be converted into a structured AI4S loop: propose, screen, simulate, stress-test, advise, decide, and iterate. The system cannot replace the judgment that comes from knowing the client, the court, the judge, opposing counsel, and the lived texture of litigation. Those judgments belong to the human lawyer. But the system can organize the search space, make the reasoning more transparent, and ensure that recommendations are not merely persuasive but governed. Its value lies not in replacing legal judgment, but in making legal judgment more informed, disciplined, auditable, and professionally defensible.

Appendix A. Implementation Artifacts

The architecture can be implemented as a compact computational workflow. One executable artifact is available for readers who want to reproduce the reference implementation:

[Open the reference implementation](#)

The artifact is not required to understand the paper. It is provided to make the design reproducible and to show how the architecture can be instantiated as code. The executable workflow is organized into nine components:

1. Setup, dependencies, model client, and parsing utilities.
2. Synthetic legal universe: jurisdiction, ethics rules, causes of action, and precedent database.
3. Case facts and client profile for *Meridian Analytics LLC v. Quantum Dynamics Corp.*.
4. Precedent engine and outcome simulator.
5. `LitigationStrategyLoop`.
6. `AttorneyAdvisorAgent`.
7. `LitigationEvaluationAgent`.
8. Orchestrator and execution entry point.
9. Final litigation report dashboard and visual audit trail.

The implementation artifact should be interpreted as an executable expression of the reference architecture. It does not represent legal advice, does not reproduce actual law, and does not create an attorney-client relationship. Its purpose is to show how AI-assisted legal strategy can be governed through explicit facts, synthetic legal rules, structured agent outputs, ethical gates, precedent-based simulation, stress tests, attorney review, settlement analysis, and decision logs.

Appendix B. Workflow Diagram

Figure B.1 provides the workflow diagram corresponding to the agent-governed AI4S litigation architecture described in Section 2. It summarizes the movement from case facts and a synthetic legal universe to theory generation, ethical screening, precedent-based simulation, stress testing, attorney review, and evaluation-agent decision.

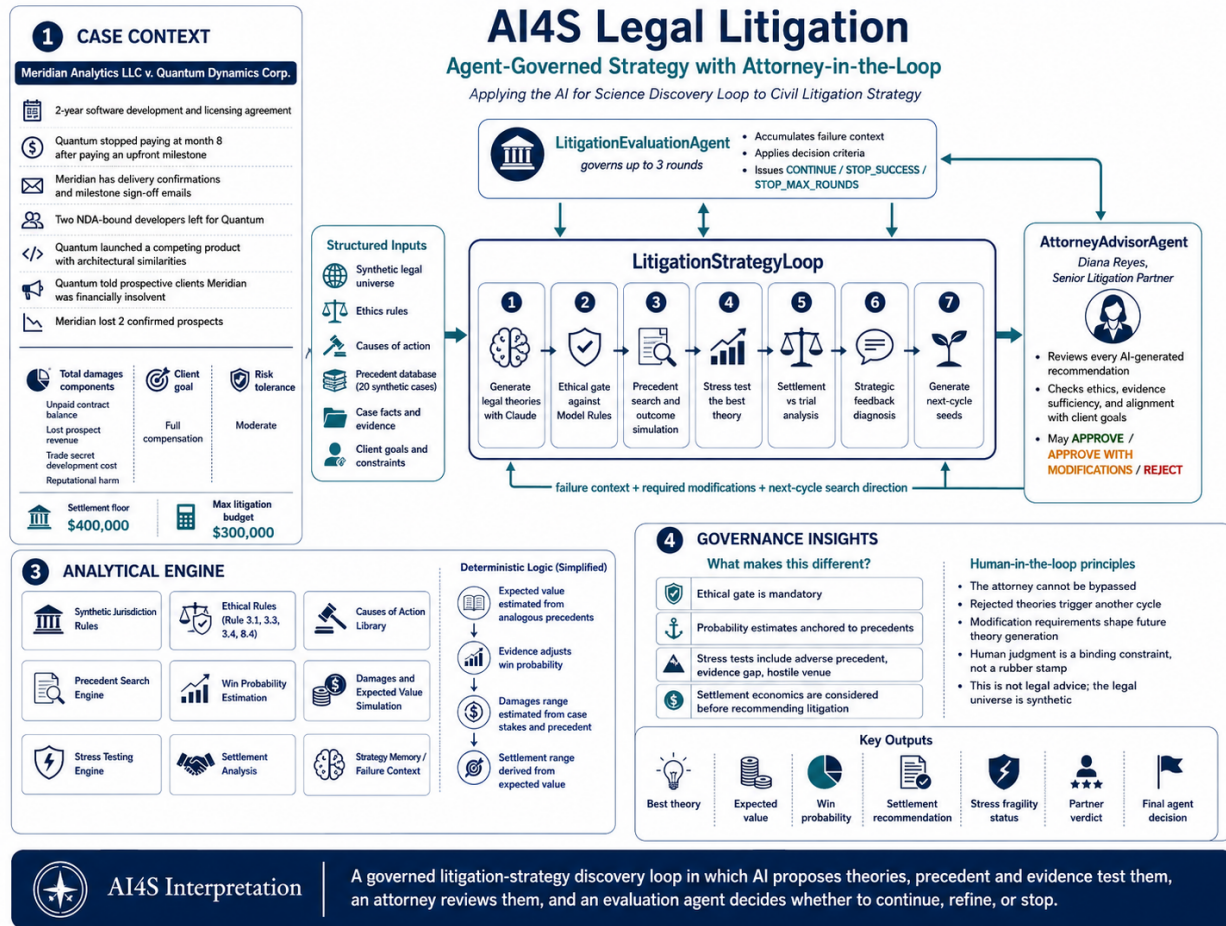


Figure B.1: Workflow diagram for the agent-governed AI4S litigation-strategy architecture.

References

- [1] H. Wang *et al.*, “Scientific discovery in the age of artificial intelligence,” *Nature*, vol. 620, pp. 47–60, 2023.
- [2] J. Jumper *et al.*, “Highly accurate protein structure prediction with AlphaFold,” *Nature*, vol. 596, pp. 583–589, 2021.
- [3] B. Burger *et al.*, “A mobile robotic chemist,” *Nature*, vol. 583, pp. 237–241, 2020.
- [4] S. Russell, *Human Compatible: Artificial Intelligence and the Problem of Control*. Penguin, 2020.
- [5] National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework*. NIST AI 100-1, 2023.
- [6] American Bar Association, “Formal Opinion 512: Generative Artificial Intelligence Tools,” Standing Committee on Ethics and Professional Responsibility, 2024.
- [7] American Bar Association, *Model Rules of Professional Conduct*. American Bar Association, 2020.
- [8] United States Courts, *Federal Rules of Civil Procedure*. Administrative Office of the U.S. Courts.
- [9] H. Surden, “Artificial intelligence and law: An overview,” *Georgia State University Law Review*, vol. 35, no. 4, pp. 1305–1337, 2019.